RICE UNIVERSITY

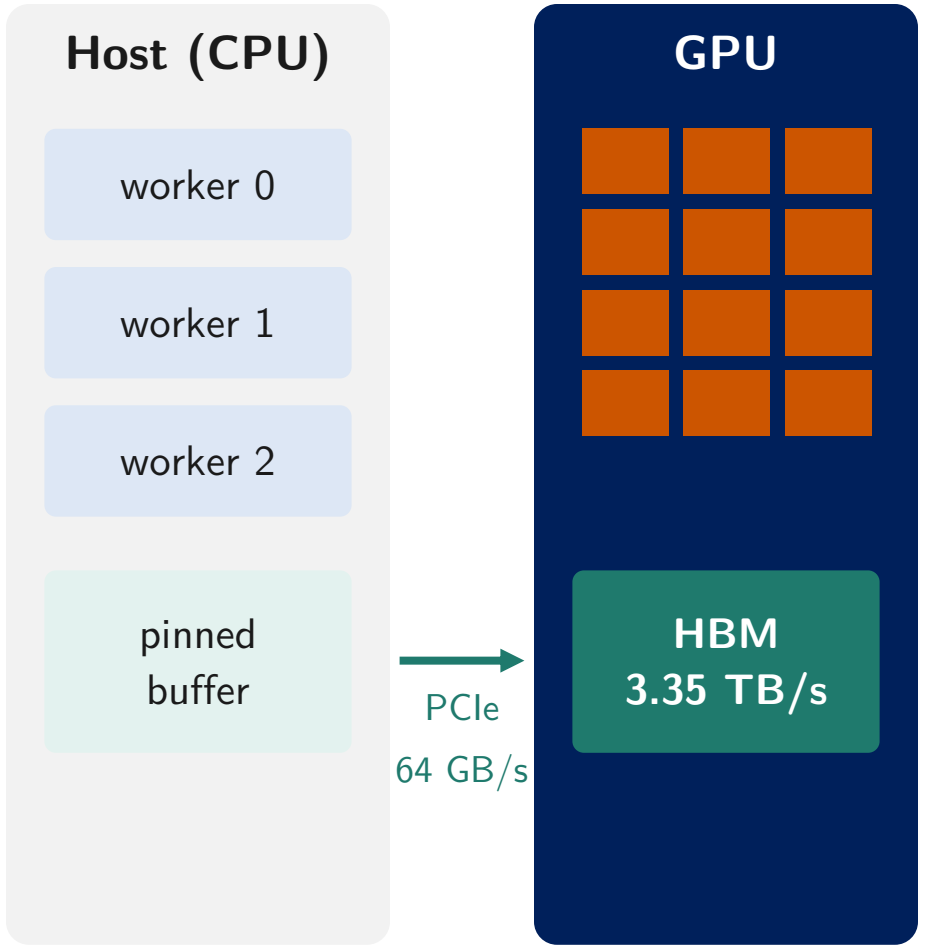
Framework and GPU Interaction

Fusion, memory, input pipelines, and precision: what the framework does around your kernels

Yuke Wang

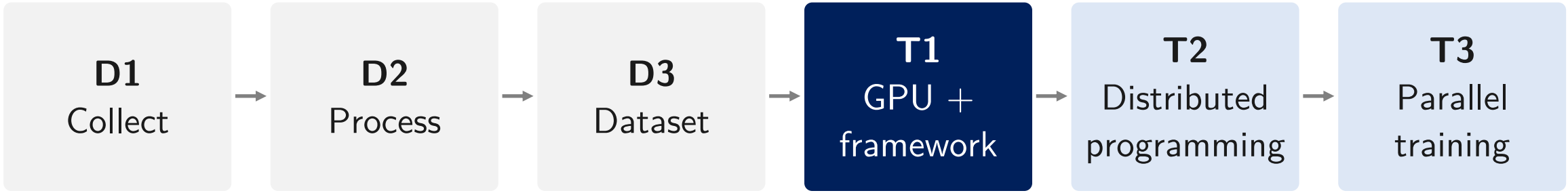
Rice University

COMP 329 · Spring 2027 · Week 10, Lecture 1



One batch: workers → pinned buffer → PCIe → HBM

Where we are in the pipeline



Week 9: tensors, dispatch, the forward pass; graphs and torch.compile

In our pipeline

Today: what the framework does **around** your kernels. **T1 due Friday**; Lab 3 due after this lecture.

Lecture 2 this week starts distributed programming: T2 and Lab 4 are released after it.

Learning goals for today

- Predict the **bytes saved** when the compiler fuses elementwise operations.
- Explain the **caching allocator**: allocated vs reserved memory, fragmentation, and OOM.
- Design an **input pipeline**: workers, pinned memory, `non_blocking` copies, streams.
- Compare **FP32, TF32, BF16, FP16, FP8** by range and precision; explain mixed precision.
- Integrate a **Triton kernel** into the forward pass and **test a bottleneck hypothesis**.

A puzzle to start

Profiler, one training step

wall-clock time: 100 ms

GPU kernels busy: 30 ms

The upgrade

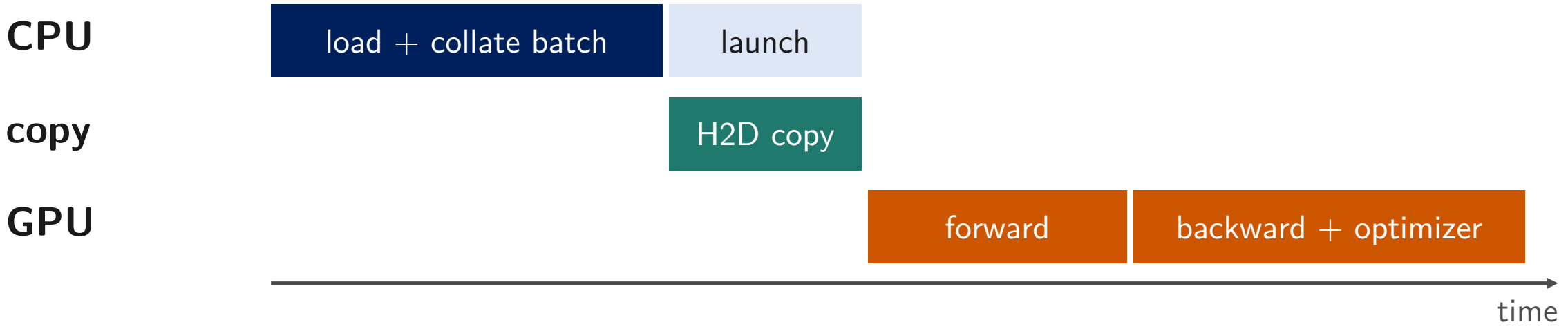
a new GPU that runs every kernel 2× faster

How long does one step take after the upgrade?

Key idea

$70 + 30 / 2 =$ **85 ms**: only 15% faster. The other 70 ms is the host, copies, and waiting — today's subject.

Anatomy of one training step



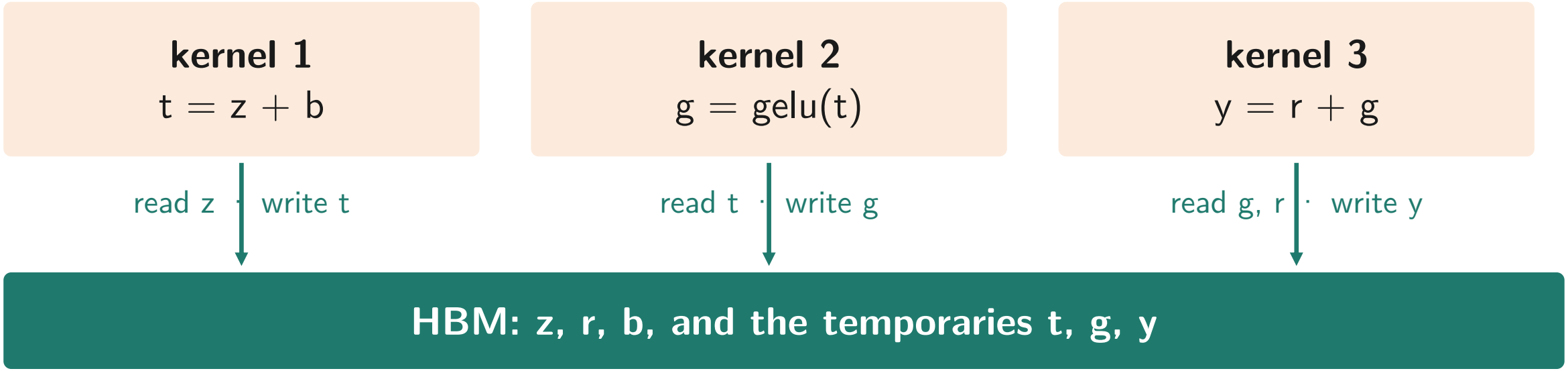
Part	Shortens
1. Fusion	the GPU lane: fewer bytes, fewer launches
2. Allocation	hidden synchronizations and OOM crashes
3. Input pipeline	the CPU and copy lanes: overlap them with compute
4. Precision	both bytes and arithmetic

Part 1

Fusion, revisited

What does the compiler fuse, and how many bytes does it save?

Eager mode: one kernel per operation



Watch out

Every temporary makes a **round trip to HBM**, and every operator pays a **launch**.

What TorchInductor fuses

Pattern	Example	Fused?
Pointwise chain (vertical)	bias $\rightarrow$ GELU $\rightarrow$ residual add	yes: one kernel
Pointwise around a reduction	softmax, layer norm, loss	yes: one kernel
Siblings reading one input	$x \cdot a$ and $x \cdot b$ from the same x	yes (horizontal)
Matrix multiply	x @ W	cuBLAS call; epilogue only with max-autotune
Across a graph break	a print() mid-model	no

Key idea

Inductor fuses the **memory-bound** glue between matmuls — exactly the kernels left of the ridge on Week 8's roofline.

Source: PyTorch docs, torch.compile and TorchInductor; Ansel et al., "PyTorch 2," ASPLOS 2024.

Bytes saved: a worked example

$$\text{one tensor: } 16{,}384 \times 4{,}096 \times 2 \text{ bytes} \approx 134 \text{ MB}$$

$$\text{unfused: } (2 + 2 + 3) \text{ passes} = 7 \times 134 \text{ MB} \approx 940 \text{ MB}$$

$$\text{fused: } (2 \text{ reads} + 1 \text{ write}) = 3 \times 134 \text{ MB} \approx 403 \text{ MB}$$

$$\text{time} \approx Q \;/\; \beta\text{:} \quad 280\ \mu s \;\rightarrow\; 120\ \mu s \quad (\approx 2.3\times)$$

Memory

Fusion leaves the FLOPs unchanged and divides the **bytes**: for memory-bound code, speedup $\approx$ passes before / passes after.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Seeing what the compiler did

```
import torch
import torch.nn.functional as F

def block(z, b, r):
    return r + F.gelu(z + b)

fast = torch.compile(block)
y = fast(z, b, r)          # first call compiles

# shell:
# TORCH_LOGS="output_code" python block.py
# -> def triton_poi_fused_add_gelu_0(...)
```

- `output_code`: the generated Triton kernels
- `graph_breaks`: where capture stopped
- `recompiles`: shapes that forced a new compile
- Profiler: kernels per step should **drop**

Source: PyTorch docs, `TORCH_LOGS` logging and the torch.compile troubleshooting guide.

Fusion in our MLP's forward pass

Fused in the forward	Passes	MLP, B = 64	Large
bias + tanh (hidden)	4 → 2	66 → 33 KB	537 → 268 MB
`log_softmax` + `nll_loss`	2 → 1	165 → 82 KB	6.59 → 3.29 GB

Large: 16,384 tokens, hidden 4,096 (BF16), vocabulary 50,257 (FP32 logits)

$$B = 64\text{: } 115 \text{ KB saved} \approx 34 \text{ ns} \quad \ll \quad 2 \text{ launches} \approx 8\ \mu s$$

$$\text{large: } 3.56 \text{ GB saved} \approx 1.06 \text{ ms per forward}$$

Memory

At B = 64 fusion saves **launches**; at scale it saves **milliseconds** of HBM traffic.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). Vocabulary: GPT-2 (Radford et al., 2019).

Check yourself

Two minutes with a neighbour.

- Three pointwise kernels, each one input and one output. Passes before and after fusion? Predicted speedup?
- Would fusing a bias add into a 4096×4096 BF16 matmul speed it up much? Why?
- Why does a `print(x)` in the middle of a model stop fusion?

Part 2

GPU memory allocation

Where does a tensor's memory come from, and why do we run out?

Why not call cudaMalloc for every tensor?

- `cudaMalloc` / `cudaFree`: **host calls into the driver**, slow next to a launch.

- `cudaFree` **waits for the whole GPU** to go idle (implicit synchronization).
- One training step creates and frees **hundreds** of temporary tensors.

Watch out

Freeing with `cudaFree` after every operator would stop the CPU from **running ahead** of the GPU.

Source: NVIDIA CUDA C++ Programming Guide, "Implicit Synchronization"; CUDA Runtime API, cudaFree.

PyTorch's caching allocator

GPU memory (80 GB) · each segment comes from one cudaMalloc



- `del t` returns the block to **PyTorch's pool**, not to the driver: no sync.
- The next request of a similar size **reuses** a cached block.
- Sizes round up to 512 B; small ($\leq$ 1 MB) and large pools; blocks split and merge.

Source: PyTorch source, c10/cuda/CUDACachingAllocator.cpp; PyTorch docs, "CUDA semantics".

Allocated versus reserved

```
x = torch.empty(2**26, device="cuda")  # 256 MiB
MB = 2**20
print(torch.cuda.memory_allocated() // MB)  # 256
print(torch.cuda.memory_reserved() // MB)   # 256
del x
print(torch.cuda.memory_allocated() // MB)  # 0
print(torch.cuda.memory_reserved() // MB)   # 256
torch.cuda.empty_cache()
print(torch.cuda.memory_reserved() // MB)   # 0
```

- **allocated**: bytes in live tensors
- **reserved**: bytes the allocator holds
- nvidia-smi $\approx$ reserved + CUDA context

Key idea

Peak usage: `reset_peak_memory_stats()` before a step, `max_memory_allocated()` after.

Source: PyTorch docs, "CUDA semantics" (memory management, pinned memory, streams).

Fragmentation



10 GB reserved, 0.5 GB per cell · teal: live tensors · white: free

$$\text{free} = 12 \text{ cells} = 6 \text{ GB}, \quad \text{largest free run} = 3 \text{ cells} = 1.5 \text{ GB}$$

```
torch.empty(2 GB)  →  CUDA out of memory
```

Remedies

Fix: consistent shapes, drop dead references, `PYTORCH_ALLOC_CONF=expandable_segments:True`.

Source: PyTorch docs, "CUDA semantics": optimizing memory usage with `PYTORCH_ALLOC_CONF`.

Reasoning about OOM before it happens

Bytes / parameter, Adam	FP32	Mixed (BF16)
Weights	4	2 + 4 FP32 master
Gradients	4	2
Adam moments m and v	8	8
Total	**16**	**16**

$$10^9 \text{ params} \times 16 \text{ bytes}$$
$$= 16 \text{ GB of state}$$
$$80 - 16 = 64 \text{ GB left}$$

Watch out

Activations scale with **batch × sequence length**. On OOM: halve the batch, accumulate gradients over two micro-batches.

Source: Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020.

Check yourself

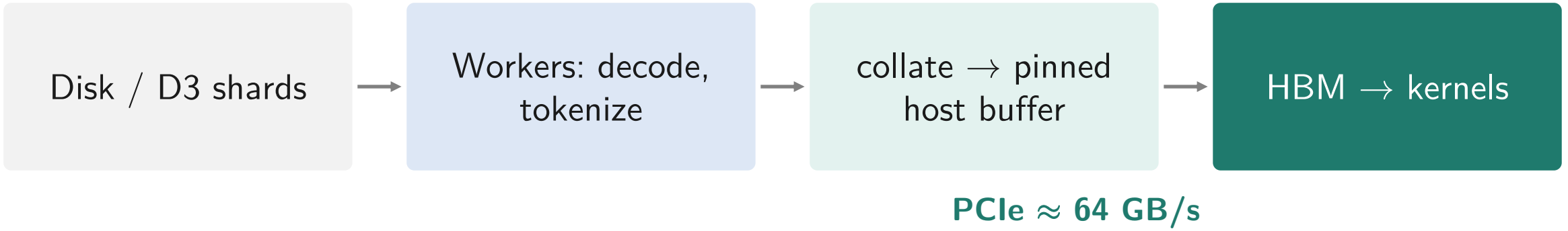
- You `del` a 1 GB tensor. What happens to `memory_allocated`, `memory_reserved`, and nvidia-smi?
- OOM: "tried to allocate 4 GiB", yet 10 GiB is "reserved but unallocated". Diagnosis?
- A 3B-parameter model, Adam, mixed precision, on one 80 GB GPU: how much is left for activations?

Part 3

Input pipelines

How does a batch get from disk to the GPU without making it wait?

The path of one batch



$$\text{HBM / PCIe} = 3.35\ \text{TB/s} \,/\, 64\ \text{GB/s} \approx 52$$

Key idea

Cross PCIe **once**, then keep data on the device; prepare the next batch **while** the GPU computes.

Source: PCI-SIG, PCIe 5.0 x16 ≈ 64 GB/s per direction (theoretical); H100 datasheet (HBM).

DataLoader: the knobs that matter

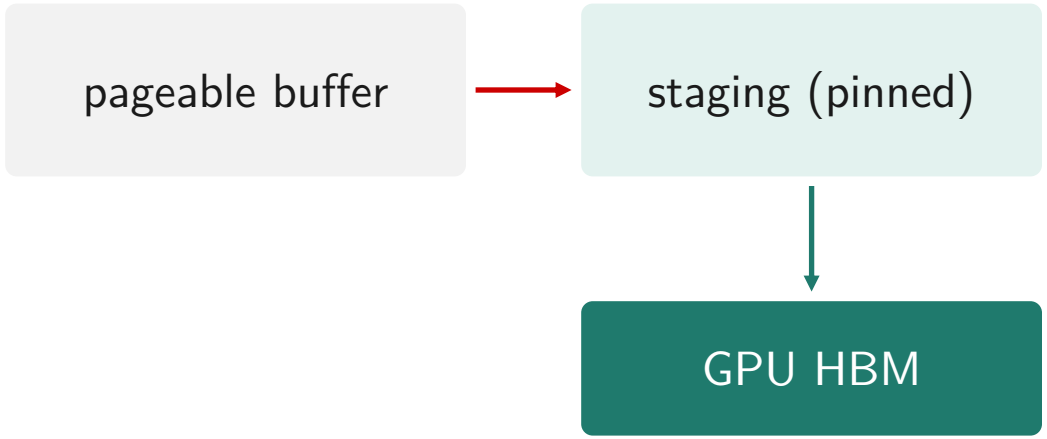
```
loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,           # processes
    prefetch_factor=2,       # batches per worker
    pin_memory=True,         # page-locked buffers
    persistent_workers=True,
)
for x, y in loader:
    x = x.to("cuda", non_blocking=True)
    y = y.to("cuda", non_blocking=True)
    train_step(x, y)
```

- Workers are **processes**: the GIL lets one thread run Python at a time.
- In flight: up to **workers** × **prefetch** batches.
- `pin_memory` makes `non_blocking` copies possible.

Source: PyTorch docs, torch.utils.data.DataLoader.

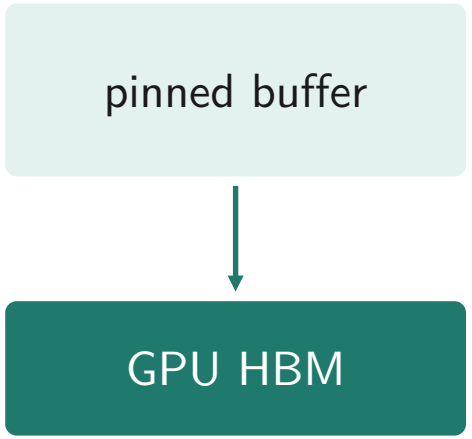
Pageable versus pinned memory

Pageable



CPU copies first; the call waits

Pinned



DMA reads it directly;
`non_blocking=True` returns at once

Watch out

`non_blocking=True` is asynchronous **only** from pinned memory. Pin the next batches, not the whole dataset.

Source: PyTorch docs, "CUDA semantics": pinned memory buffers; CUDA C++ Programming Guide.

How long does a copy take?

$$T(n) \approx \alpha + n \,/\, \beta, \quad \beta \approx 64 \text{ GB/s (PCIe 5.0)}$$

$$\text{images uint8: } 38.5 \text{ MB} \;\rightarrow\; 0.60 \text{ ms}$$

$$\text{same batch as FP32: } 154 \text{ MB} \;\rightarrow\; 2.4 \text{ ms}$$

$$\text{our tokens: } 64 \times 8 \times 8 \text{ bytes} = 4 \text{ KB} \;\rightarrow\; 64 \text{ ns } (\alpha \text{ wins})$$

Memory

Send **compact types** (uint8, int32 token ids) and convert on the GPU; batch small copies together.

Source: PCI-SIG, PCIe 5.0 x16 ≈ 64 GB/s per direction (theoretical); H100 datasheet (HBM).

Overlapping copies with compute

one stream



step = copy + compute

copy stream



compute



step = max(copy, compute)

$$\text{pipelined: } \text{step} = \max(\text{load}, \text{copy}, \text{compute}), \text{ not the sum}$$

Key idea

With overlap, the slowest stage sets the pace; the others become **free**.

Prefetching on a side stream

```
side = torch.cuda.Stream()
main = torch.cuda.current_stream()
def prefetch(cpu_batch):
    with torch.cuda.stream(side):
        return [t.to("cuda", non_blocking=True)
                for t in cpu_batch]

nxt = prefetch(next(it))
for _ in range(steps):
    main.wait_stream(side)      # copy done?
    x, y = nxt
    x.record_stream(main); y.record_stream(main)
    nxt = prefetch(next(it))    # overlaps step
    train_step(x, y)
```

- `wait_stream`: GPU-side wait, the CPU keeps going.
- `record_stream`: don't reuse this memory while `main` still reads it.
- Enqueue the next copy **before** the step.

Source: PyTorch docs, "CUDA semantics" (memory management, pinned memory, streams).

Hidden synchronizations

Code	Why the CPU must wait
`loss.item()`, `print(loss)`	Needs the value on the host
`t.cpu()`, `t.tolist()`	Device $\rightarrow$ host copy of a result
`if loss > 10:`	Python branch on a GPU value
`x[mask]`, `torch.nonzero(x)`	Output size depends on the data

Watch out

Log every N steps, keep running sums on the GPU, and find culprits with `torch.cuda.set_sync_debug_mode("warn")`.

Source: PyTorch docs, "CUDA semantics: asynchronous execution"; `torch.cuda.set_sync_debug_mode`.

Check yourself

- The GPU idles at the start of every step and `num_workers=0`. What is happening?
- `x.to("cuda", non_blocking=True)` takes as long as a blocking copy. Most likely cause?
- Load 5 ms, copy 2 ms, compute 8 ms per step: serial vs pipelined step time?

Part 4

Precision

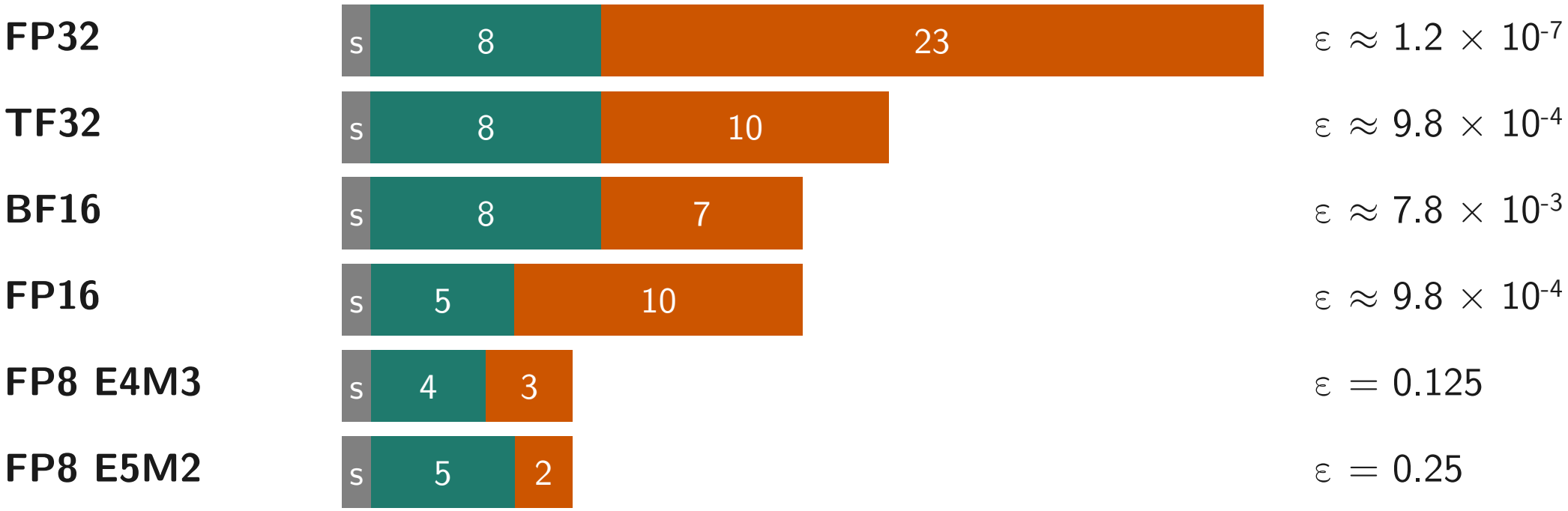
How few bits can we compute with, and what breaks when we try?

Floating-point formats, bit by bit

$$x = \pm\, 1.f \times 2\hat{}(e - \text{bias})$$

$$\text{exponent bits} \to \text{range}; \quad \text{mantissa bits } M \to \varepsilon = 2\hat{}(-M)$$

grey: sign · teal: exponent · orange: mantissa · one cell = one bit



Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (TF32, BF16); Micikevicius et al., "FP8 Formats for Deep Learning," 2022.

Range, precision, and speed

Format	Bits (E, M)	Largest	ε	H100 peak
FP32	8, 23	3.4×10^{38}	1.2×10^{-7}	67 TFLOP/s
TF32 (tensor core)	8, 10	3.4×10^{38}	9.8×10^{-4}	495 TFLOP/s
BF16	8, 7	3.4×10^{38}	7.8×10^{-3}	989 TFLOP/s
FP16	5, 10	65,504	9.8×10^{-4}	989 TFLOP/s
FP8 E4M3 / E5M2	4, 3 / 5, 2	448 / 57,344	0.125 / 0.25	1,979 TFLOP/s

Compute

Halving the bits **doubles** the tensor-core peak and **halves** the bytes; the ridge moves from 20 to 295 FLOP/byte.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). Dense tensor-core values.

Two ways low precision fails

Underflow (range)
gradient $g = 10^{-8}$
FP16: rounds to **0**
BF16, FP32: kept

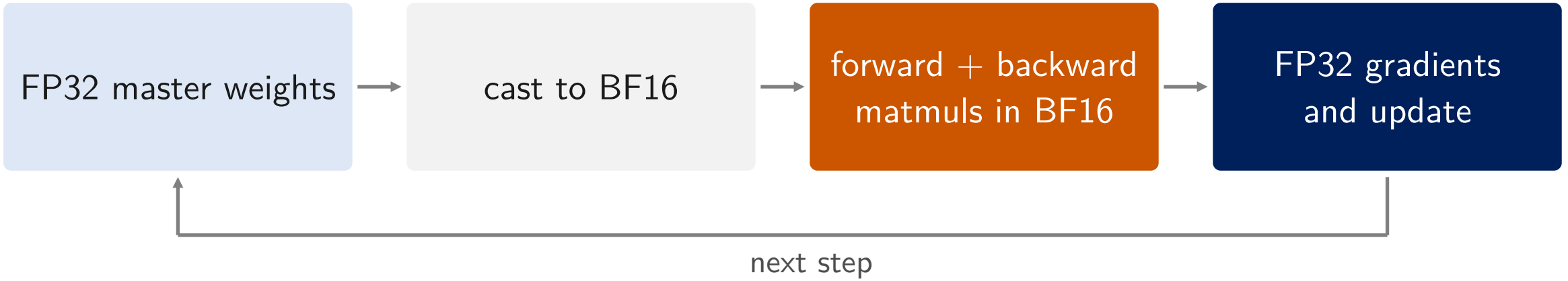
Swamping (precision)
$w + \eta \cdot g = 1 + 10^{-4}$
BF16, FP16: $= \mathbf{1}$ (update lost)
FP32: kept

$$\text{lost when } |\eta \cdot g| < \varepsilon \cdot |w| \,/\, 2\text{:} \quad \text{BF16 needs } \eta \cdot g > 0.004 \text{ at } w = 1$$

Check

Fixes: keep **FP32 master weights** (swamping) and **scale the loss** in FP16 (underflow).

Mixed precision: the recipe



- Matmuls and convolutions: **BF16** on tensor cores.
- Softmax, log, sums, loss, normalization: kept in **FP32**.
- Weight update in **FP32**: small steps are not swamped.

Source: Micikevicius et al., "Mixed Precision Training," ICLR 2018; PyTorch AMP docs.

Mixed precision in PyTorch

```
model = MLP().cuda()              # FP32 weights
opt = torch.optim.AdamW(model.parameters())

for x, y in loader:
    with torch.autocast("cuda",
                        dtype=torch.bfloat16):
        logits = model(x)         # BF16 matmuls
        loss = F.cross_entropy(logits, y)  # FP32
    loss.backward()               # FP32 .grad
    opt.step()                    # FP32 update
    opt.zero_grad(set_to_none=True)
```

- Wrap **forward and loss** only.
- Parameters and `.grad` stay **FP32**.
- BF16: **no** gradient scaler needed.

Key idea

Autocast picks the dtype **per operator**: matmuls go low, reductions stay FP32.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., ICLR 2018.

FP16 needs loss scaling; BF16 does not

$$\partial(S \cdot L) \;/\; \partial w = S \cdot \partial L \;/\; \partial w$$

$$10^{-8} \times 2^{16} \approx 6.6 \times 10^{-4}$$

- Start S $= 2^{16}$; on inf/NaN **skip the step** and halve S.
- After 2,000 clean steps, double S.

```
scaler = torch.amp.GradScaler("cuda")
with torch.autocast("cuda",
                    dtype=torch.float16):
    loss = loss_fn(model(x), y)
scaler.scale(loss).backward()  # S · loss
scaler.step(opt)    # unscale; skip if inf
scaler.update()     # adjust S
```

Check

BF16 has FP32's exponent range: gradients do not underflow, so it needs **no scaler**.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., ICLR 2018.

Check yourself

- An activation reaches 70,000. Which of FP16 and BF16 can store it? Why?
- Pure BF16 training, $w = 1$, update $\eta \cdot g = 10^{-3}$. What happens? How does AMP avoid it?
- Why does BF16 training not need a `GradScaler`?

IN-CLASS BUILD

Integrate and profile

Pairs: put your T1 kernel in the forward pass, then predict and measure.

Step 1: wrap the kernel as an operator

```
from torch.library import triton_op, wrap_triton

@triton_op("t1::bias_tanh", mutates_args={})
def bias_tanh(z: Tensor, b: Tensor) -> Tensor:
    h = torch.empty_like(z)
    n, H = z.numel(), z.shape[-1]
    grid = (triton.cdiv(n, 1024),)
    wrap_triton(bias_tanh_kernel)[grid](
        z, b, h, n, H, BLOCK=1024)
    return h

# in the MLP's forward, forward-only runs:
h = bias_tanh(x @ W1, b1)  # was tanh(x@W1 + b1)
```

- `triton_op`: a **named operator** with a schema.
- `wrap_triton`: the launch the compiler can see.
- **No backward**: training keeps PyTorch's layers.

Source: PyTorch docs, `torch.library.triton_op` and `wrap_triton` (PyTorch 2.6+).

Step 2: validate the forward

```
from torch.testing import assert_close
SHAPES = [(64, 64), (1, 64), (999, 64), (5, 33)]
for B, H in SHAPES:
    for dt in (torch.float32, torch.bfloat16):
        kw = dict(device="cuda", dtype=dt)
        z = torch.randn(B, H, **kw)
        b = torch.randn(H, **kw)
        assert_close(bias_tanh(z, b),
                     torch.tanh(z + b))

with torch.inference_mode():  # no autograd
    logits = model(x)         # kernel inside
```

dtype	rtol	atol
FP32	1.3×10^{-6}	10^{-5}
BF16	1.6×10^{-2}	10^{-5}

- Edge shapes: $\mathbf{B = 1}$, size not a multiple of BLOCK, odd H.
- `inference_mode`: time the **forward only**.

Source: PyTorch docs, `torch.testing.assert_close` (default tolerances); `torch.inference_mode`.

Step 3: opcheck, then compile

```
from torch.library import opcheck
z = torch.randn(64, 64, device="cuda")
b = torch.randn(64, device="cuda")
opcheck(bias_tanh, (z, b))   # raises if wrong

fast = torch.compile(model, fullgraph=True)
with torch.inference_mode():
    fast(x)        # a graph break is an error

# shell: list any graph breaks
# TORCH_LOGS="graph_breaks" python t1.py
```

- `opcheck`: schema, fake-tensor, and compile-path tests.
- `fullgraph=True`: **no graph break**, or an error.
- Then compare **eager vs compiled** forward time.

Source: PyTorch docs, `torch.library.opcheck`; `torch.compile` (`fullgraph`); `TORCH_LOGS`.

Step 4: test a hypothesis, not a hunch

Hypothesis	Prediction	Measurement	Conclusion
"The step is launch-bound."	GPU busy $< 50\%$; batch $\times 2 \rightarrow$ time $\approx$ same	profiler + one intervention	accept or reject; next test

Key idea

A profile shows **where** time goes; an **intervention** that changes one thing tests **why**.

Measuring with torch.profiler

```
from torch.profiler import profile, schedule
from torch.profiler import record_function
from torch.profiler import ProfilerActivity as A

with profile(activities=[A.CPU, A.CUDA],
             schedule=schedule(wait=1, warmup=2,
                               active=5)) as prof:
    for _ in range(8):
        with record_function("data"):
            x, y = next(it)
        train_step(x, y)
        prof.step()
print(prof.key_averages().table(
    sort_by="device_time_total", row_limit=10))
```

- Warm-up steps hide **compile** and allocator costs.
- `record_function` labels the data wait.
- `prof.export_chrome_trace(...)` $\rightarrow$ Perfetto timeline.

Check

GPU busy fraction $= \Sigma$ kernel time / wall time.

Source: PyTorch recipe "PyTorch Profiler"; torch.profiler documentation.

Exercise: predict before you profile

Hypothesis	GPU busy	Batch × 2	Cached batch
H1: launch-bound			
H2: data loading starves the GPU			
H3: kernel-bound (control)			

USE

Batch 64, our T1 MLP, one GPU.

Fill in: low / high, ≈ same / ≈ 2×, faster / no change.

STRETCH

Which single intervention best separates H1 from H2?

Exercise: solutions

Hypothesis	**GPU busy**	**Batch × 2**	**Cached batch**
H1: launch-bound	low, gaps	$\approx$ same time	no change
H2: data starves the GPU	low, idle at start	$\approx 2\times$ (loading)	**much faster**
H3: kernel-bound	$\approx 100\%$	$\approx 2\times$	no change

In our pipeline

The **cached-batch** intervention separates H1 from H2. For the T1 MLP expect **H1** — Week 8's overhead-bound prediction.

Key takeaways

- **Fusion** keeps temporaries on chip: speedup $\approx$ HBM passes before $\div$ passes after.
- The **caching allocator** avoids `cudaFree` syncs; reserved $\geq$ allocated; fragmentation causes OOM.
- Training state costs $\approx$ **16 bytes per parameter** with Adam, even in mixed precision.
- **Workers + pinned memory + streams** make step time max(load, copy, compute), not the sum.
- **Exponent bits = range, mantissa bits = precision**; BF16 needs no loss scaling.
- Put kernels in the **forward pass**, validate in **FP32 and BF16**; test hypotheses by **intervention**.

Next time, and readings

When	What
Friday (this week)	T1 due: kernel, integration, compile vs eager, profiling analysis
After this lecture	Lab 3 due
Lecture 2 (this week)	Distributed programming foundations: ranks, messages, collectives
After Lecture 2	T2 and Lab 4 released

READINGS

PyTorch docs: “CUDA semantics” and “Automatic Mixed Precision”.

Micikevicius et al., “Mixed Precision Training,” ICLR 2018.